

Data Structures

Dilip Maripuri
Associate Professor
Department of Computer Applications

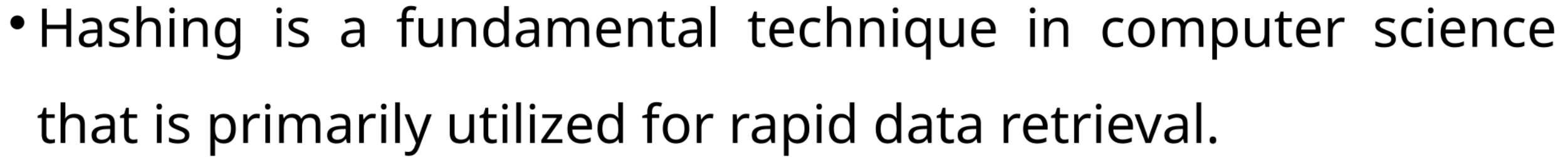


Data Structures

Session Outline



Hashing





Data Structures

Hashing



- **Definition and Purpose**

- **Hashing**

- Involves transforming input data (such as strings, files, or objects) into a compressed, fixed-size output known as a hash code, or simply "hash."

- **Primary Use**

- The generated hash is used for quick data indexing and retrieval in various applications.



Data Structures

Hashing



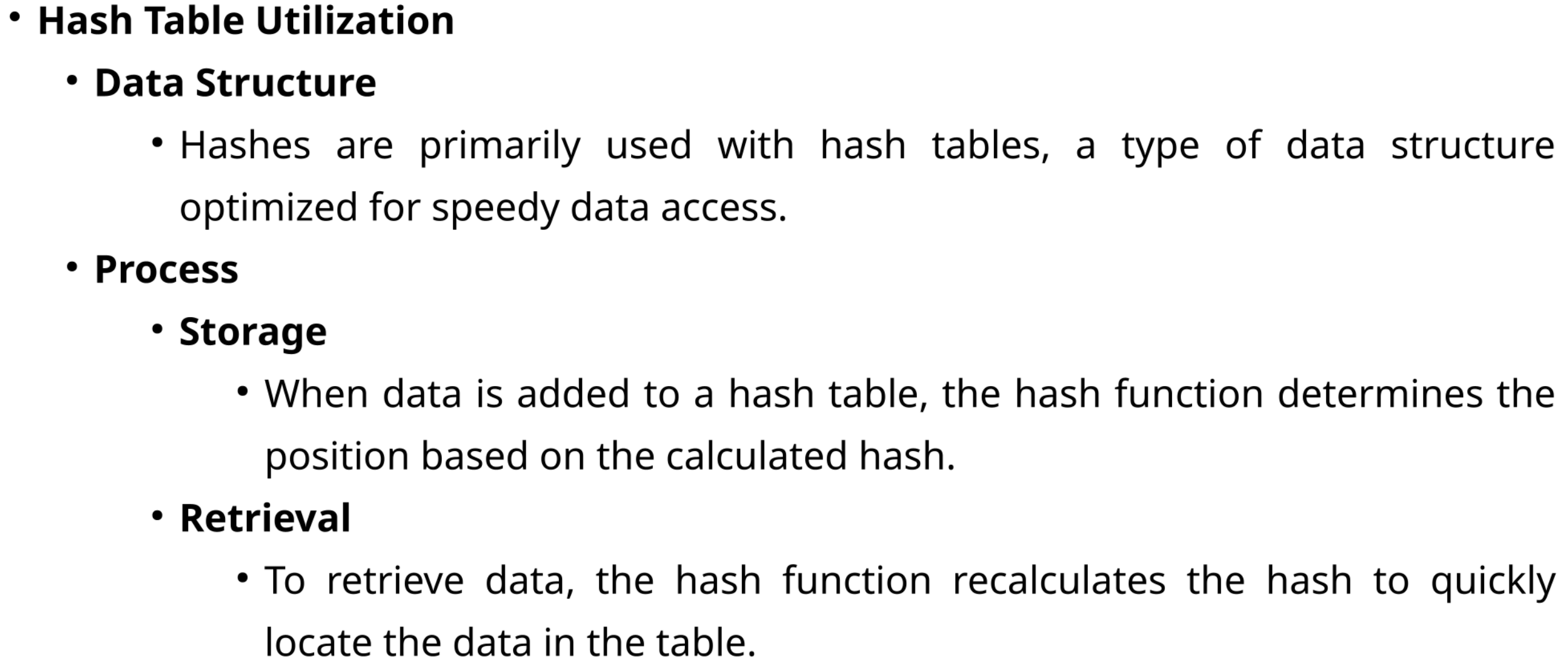
- **Hash Function**

- **Role**

- A hash function calculates a hash by processing input data. This hash acts as a unique identifier for the data.

- **Output**

- It produces a short, fixed-size value from potentially large or complex input data.





Data Structures

Hashing



- **Efficiency and Performance**

- **Speed**

- Hashing allows for near constant time complexity in data retrieval, making it highly efficient irrespective of the data size.

- **Application**

- Essential for fast-paced applications needing quick data access, such as database indexing and caching.



- Each unique piece of data ideally corresponds to a unique hash, reducing retrieval times.

- Hashes provide a compact representation of data, simplifying processes and improving performance in data management systems.

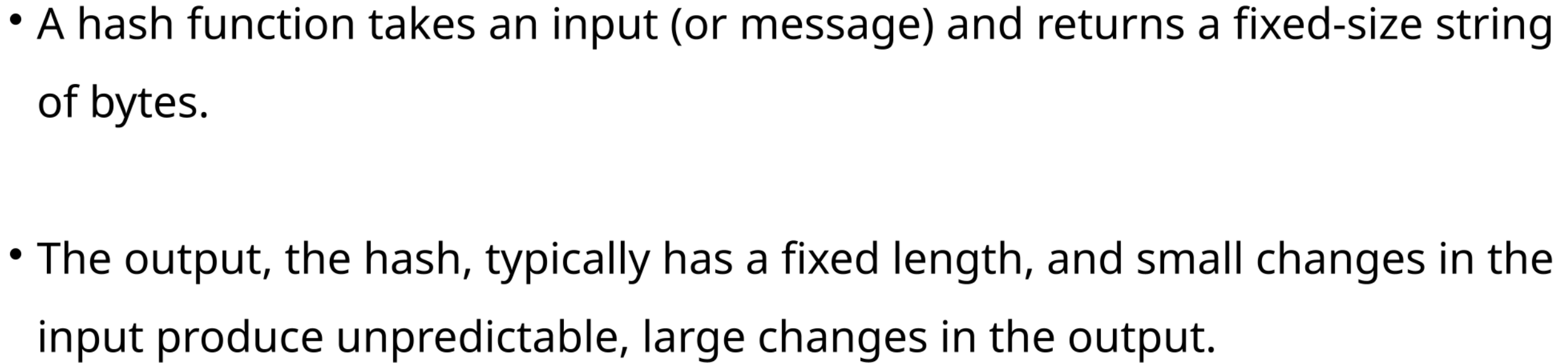


Data Structures

Hashing



- The main purposes of hash functions in computing include
 - **Speed**
 - Hash functions enable very fast data retrieval, almost independent of the size of the dataset.
 - **Uniformity**
 - Good hash functions distribute data evenly across the hash table, minimizing collisions (where different inputs produce the same hash).
 - **Determinism**
 - The same input will always produce the same hash.
 - **Efficiency**
 - Hash functions should be easy to compute while still achieving good distribution and minimal collisions.





Data Structures

Hashing



- **Basic Hash Functions**
 - **Division-Remainder Method**
 - A simple approach where the hash is calculated as the remainder of the input divided by the hash table size.
 - For example, for an input `x` and table size `m`, the hash is $h(x) = x \bmod m$.



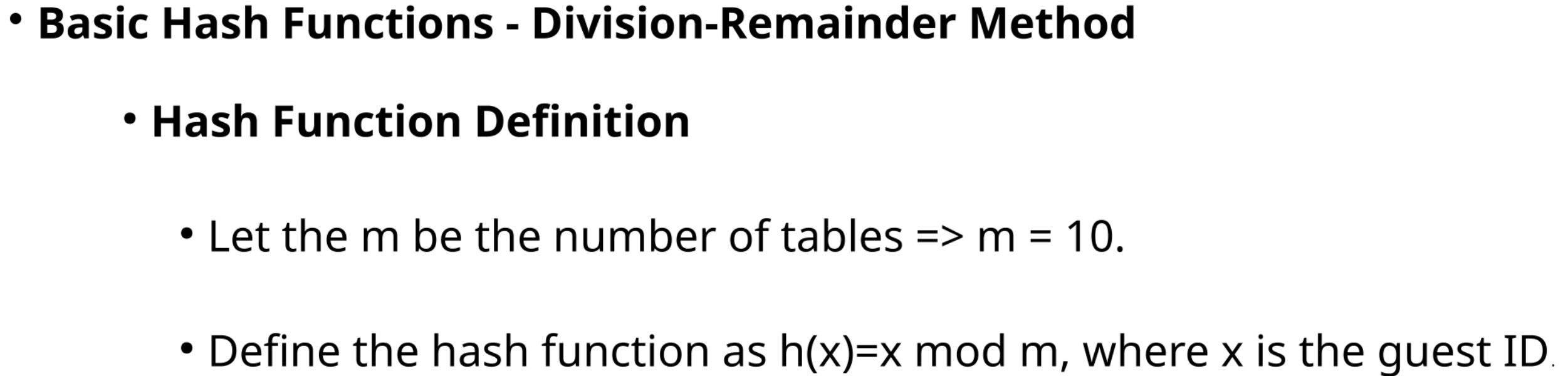
Data Structures

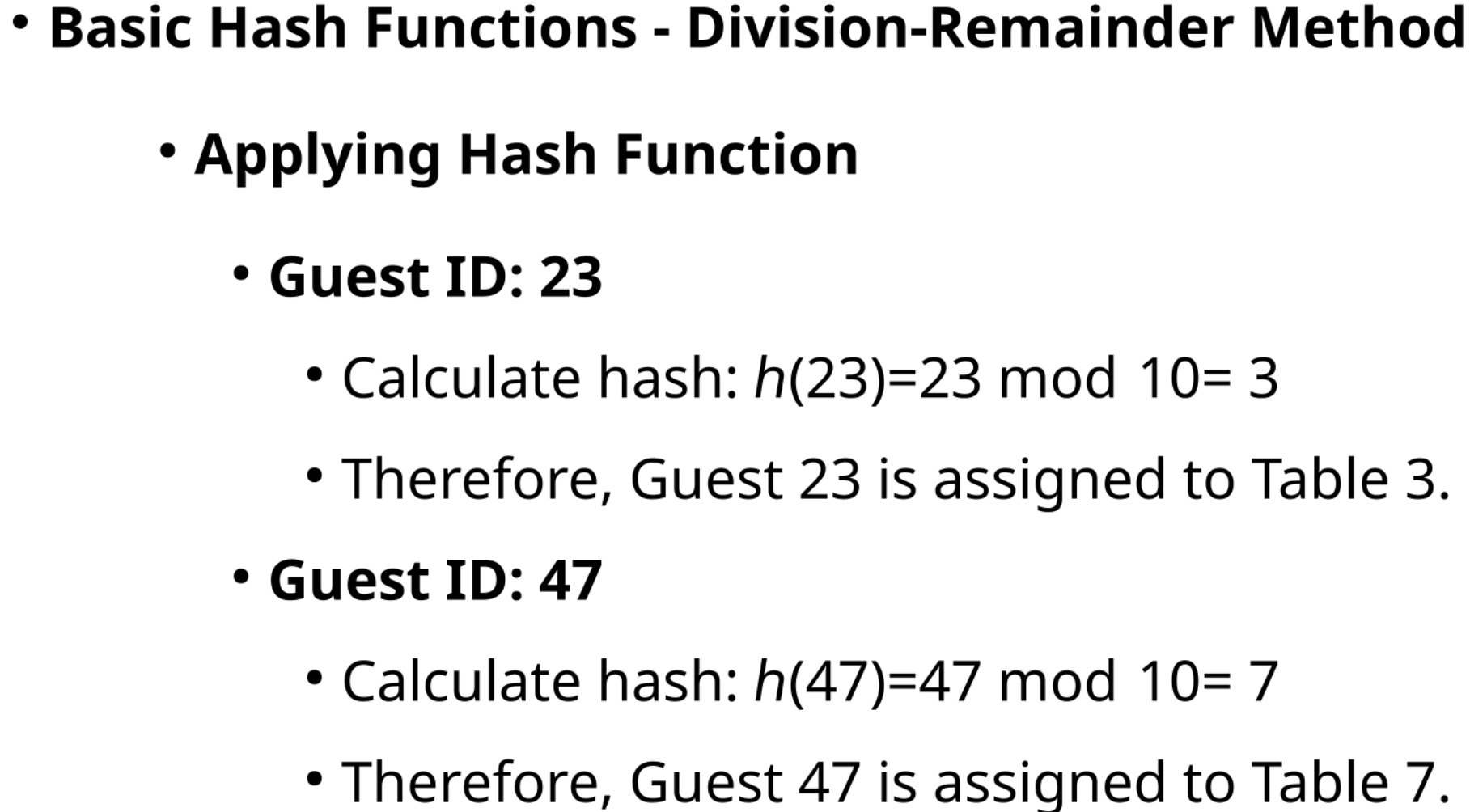
Hashing



- **Basic Hash Functions - Division-Remainder Method**

- Suppose you are developing a system to manage the seating arrangement of guests by their ID number at a banquet, and you want to use hashing for quick look-up of where each guest should be seated.
- The banquet hall has 10 tables, so you decide to use the Division-Remainder Method to determine which table each guest should sit at based on their unique guest ID.







- Compiled by Dilip Maripuri



- This method involves multiplying the input by a constant, taking the fractional part of the result, and then multiplying by the table size.
- The hash is then the floor of this value.
- Its less sensitive to patterns in the data than the division method.



- Imagine you are designing a system to allocate tasks to different servers based on task IDs, and you want to distribute these tasks evenly across a fixed number of servers using hashing.
- There are 5 servers available.



Data Structures

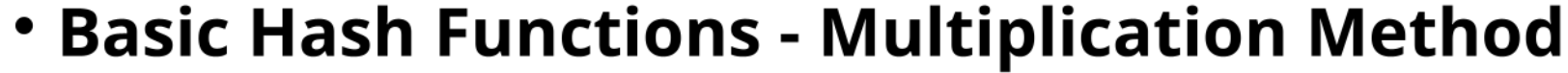
Hashing



- **Basic Hash Functions - Multiplication Method**

- **Hash Function Definition**

- Let the m be the number of servers $\Rightarrow m = 5$.
- Choose a constant A which is not too large or too small; typically, it is an irrational number fraction.
- For simplicity, let's use $A = 0.6180339887$
 - the fractional part of the golden ratio, commonly used in such scenarios.



- The choice of an irrational number helps in breaking up any regular patterns in the input data because the multiplication of a regular pattern with an irrational number results in outputs that are more spread out over the number line



Data Structures

Hashing



- **Basic Hash Functions - Multiplication Method**

- **Applying the Hash Function**

- Task ID: 12345
 - Calculate hash
 - Multiply the input by the constant $12345 \times 0.6180339887 \approx 7629.629590501$
 - Take the fractional part of the result
 - $7629.629590501 - 7626 = 0.629590501$
 - Multiply by the table size
 - $0.629590501 \times 5 \approx 3.147952505$
 - Take the floor of this value to get the hash

$\text{floor}(3.147952505) = 3$
 - Therefore, Task 12345 is assigned to Server 3.



Data Structures

Hashing

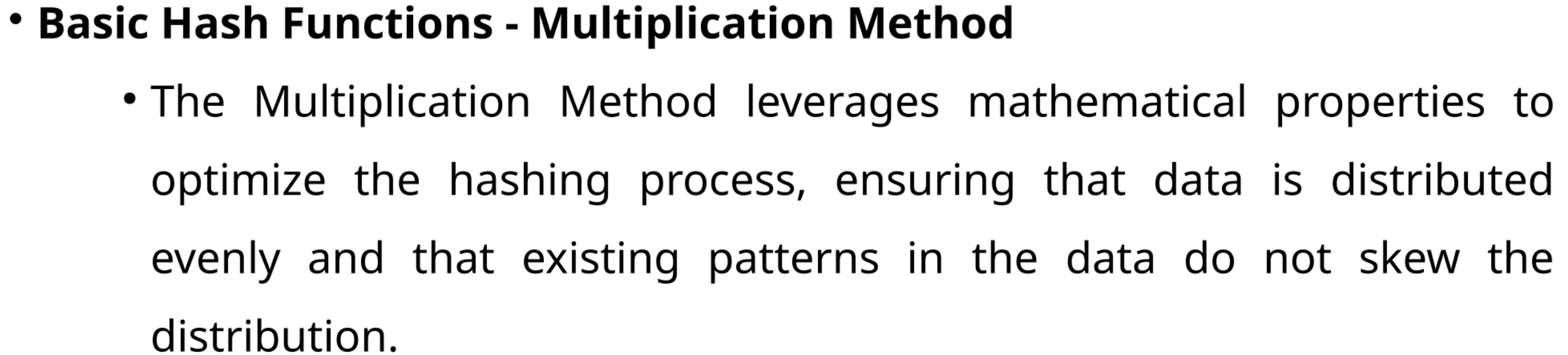


- **Basic Hash Functions - Multiplication Method**

- This focus on the fractional part is key to why the Multiplication Method is effective at distributing entries evenly.
- By eliminating the integral part of the product, the method prevents the original magnitude of inputs from influencing their hash values directly.



- This is particularly beneficial when the input data may have existing numerical or structural patterns that could otherwise lead to clustering in certain slots of a hash table.
- For instance, if task IDs are sequential or have specific numeric patterns, the Multiplication Method helps in ensuring these patterns do not translate directly to clustering in the hash table.





Data Structures

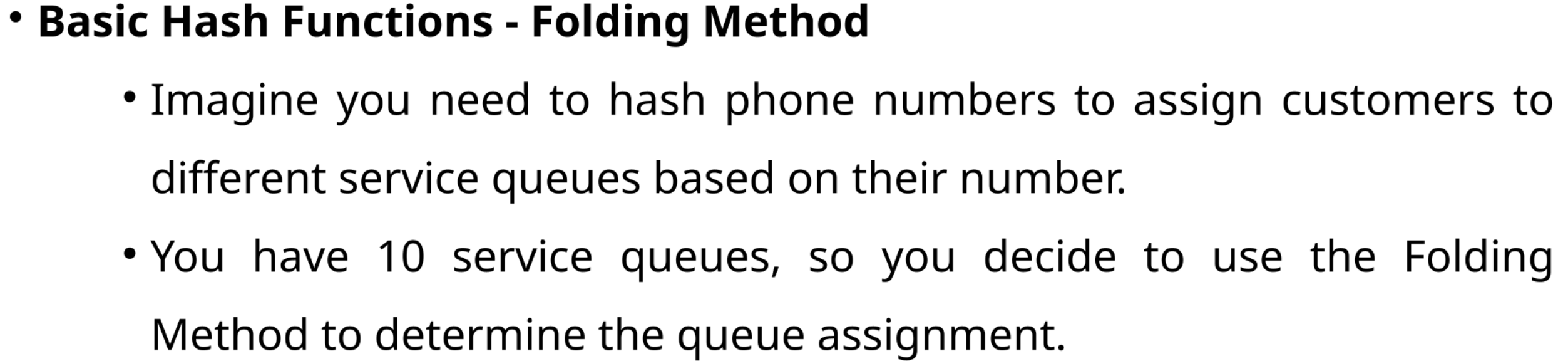
Hashing

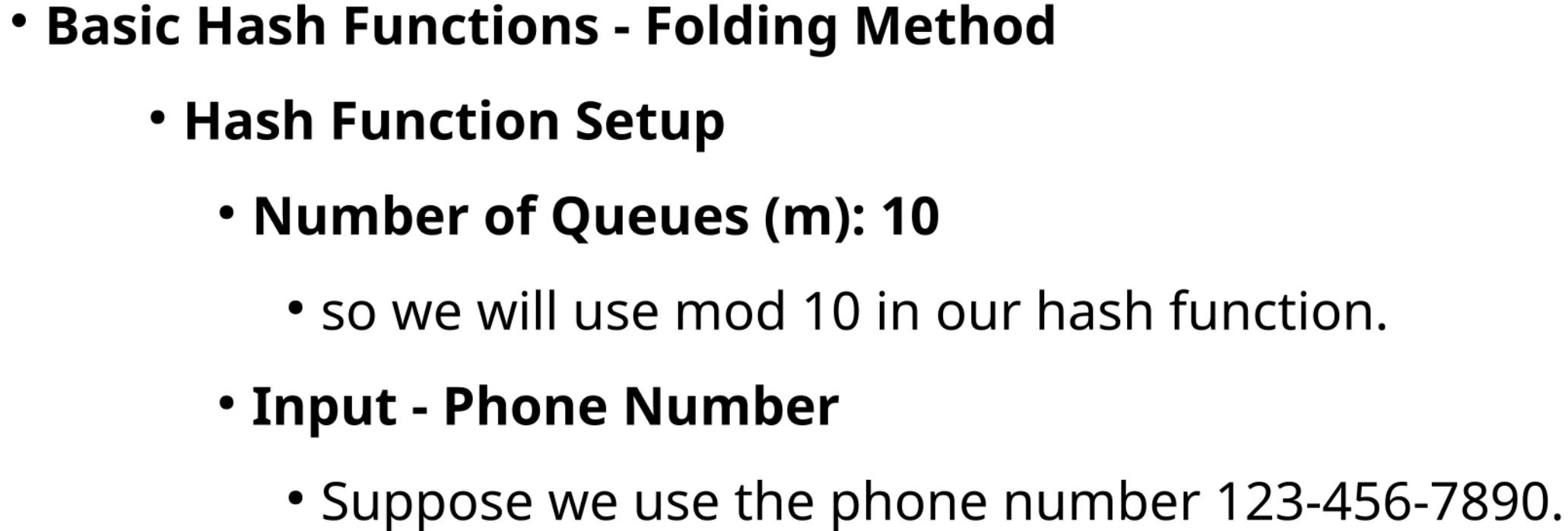


- **Basic Hash Functions**

- **Folding Method**

- This method splits the input into several parts, adds them together, and then takes the modulo of the result with the table size.
 - Its useful for hashing functions where the input might be significantly longer than the desired hash outputs.







Data Structures

Hashing



- **Basic Hash Functions - Folding Method**
 - **Steps for Hashing**
 - **Splitting the Input**
 - Break the phone number into segments of equal length: 123, 456, 789, 0 (the last segment can be shorter).
 - **Converting Segments**
 - Convert these segments into integers: 123, 456, 789, 0.
 - **Adding the Segments**
 - Sum these numbers: $123 + 456 + 789 + 0 = 1368$.
 - **Applying Modulo**
 - Take the modulo of the sum with the table size: $1368 \bmod 10 = 8$



Data Structures

Hashing



- **Basic Hash Functions - Folding Method**
 - **Queue Assignment**
 - Based on the hash, the customer with the phone number 123-456-7890 would be assigned to Queue 8.



- This involves squaring the input, then extracting some portion of the resulting number (often the middle part) as the hash.
- It can be effective but depends heavily on the input data characteristics.

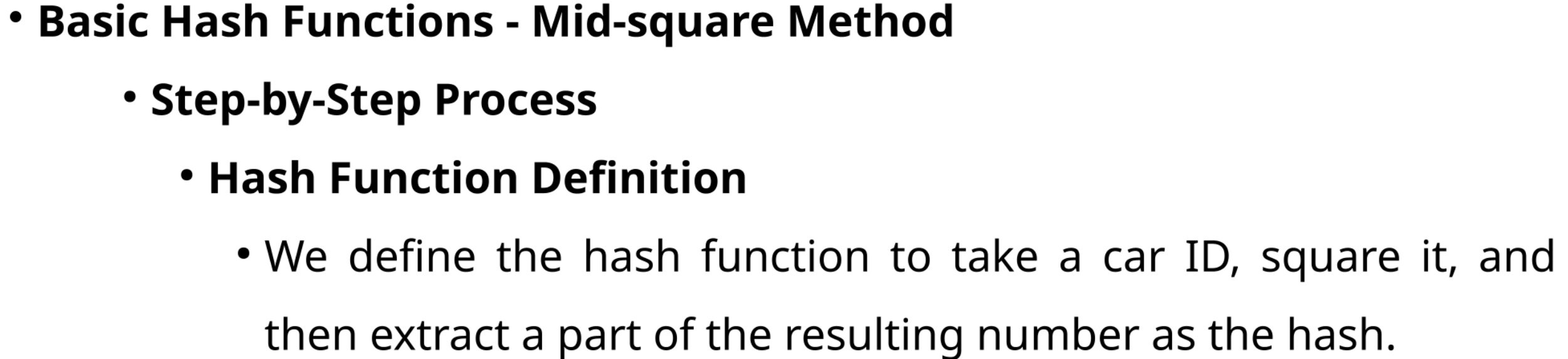


Data Structures

Hashing

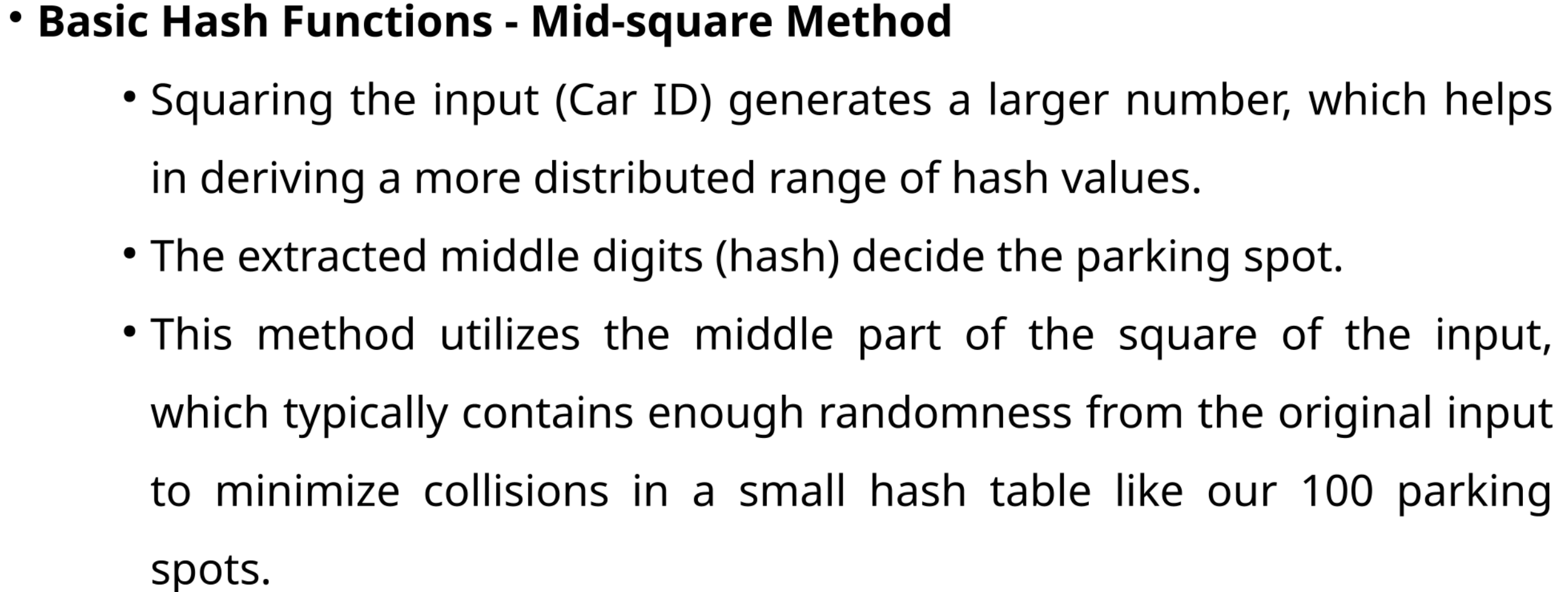


- **Basic Hash Functions - Mid-square Method**
 - Imagine we are tasked with assigning parking spaces to cars in a large parking garage.
 - Each car has a unique identification number, and we want to use hashing to determine which parking spot to assign each car efficiently.
 - We decide to use the Mid-square Method to generate a hash code from each car's ID to determine the parking spot number.





- **Basic Hash Functions - Mid-square Method**
 - **Step-by-Step Process**
 - **Applying the Hash Function**
 - Suppose the parking garage has 100 spots, so we need a two-digit result from our hash function.
 - Car ID: 1234
 - **Calculation**
 - Square the ID: $1234^2 = 1522756$
 - **Extract the middle two digits** to form the hash: from 1522756, take 27.
 - So, the hash (parking spot number) for Car ID 1234 is 27.





Data Structures

Hashing



- **Basic Hash Functions - Mid-square Method**
 - This hashing technique, while dependent on the nature of input values, provides an easy and effective way to distribute items (in this case, cars) across a fixed number of categories (parking spots), ensuring an efficient use of space and quick retrieval based on ID numbers.

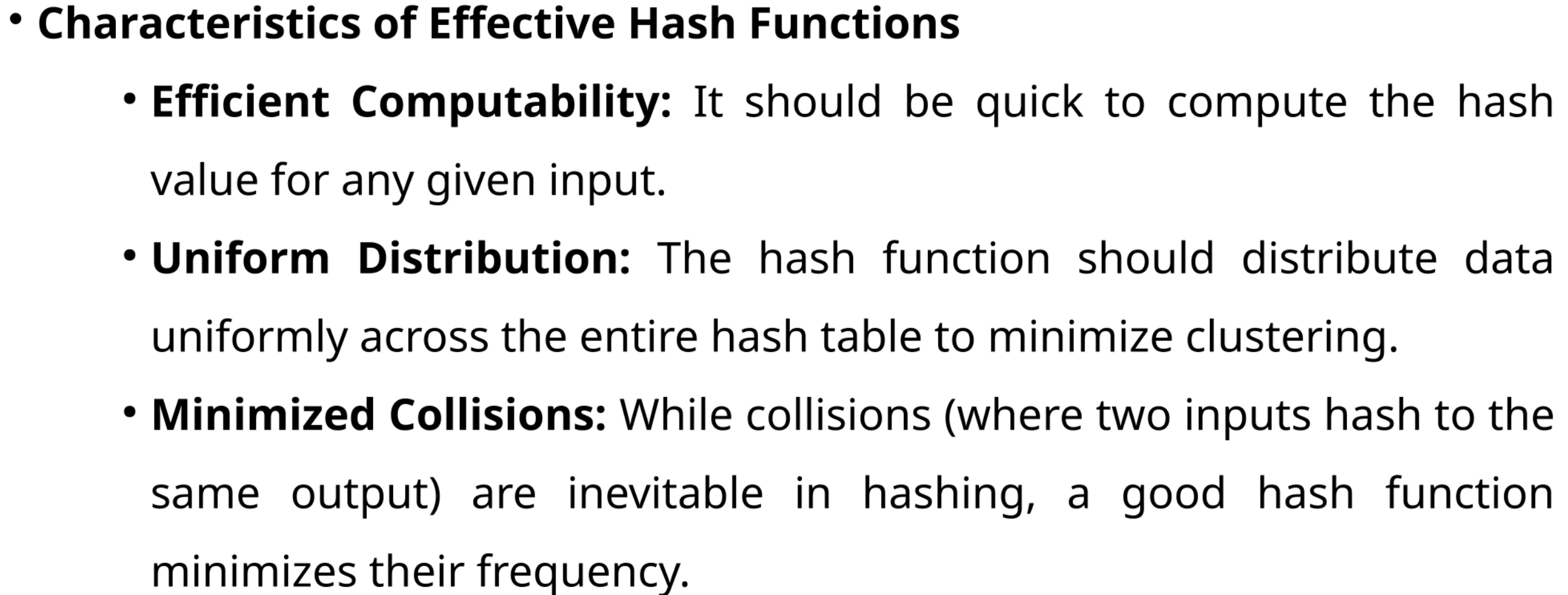


Data Structures

Hashing



- **Basic Hash Functions - Mid-square Method**
 - This hashing technique, while dependent on the nature of input values, provides an easy and effective way to distribute items (in this case, cars) across a fixed number of categories (parking spots), ensuring an efficient use of space and quick retrieval based on ID numbers.





Data Structures

Hashing



- **Characteristics of Effective Hash Functions**
 - **Avalanche Effect:** A small change in the input should cause a significant change in the output hash, making the output appear random and uniformly distributed.
 - **Irreversibility:** Especially in cryptographic hash functions, it should be computationally infeasible to reverse the hash function to find the original input.



Data Structures

Hash Functions - Summary



Basic Concept

Method

Description

Division-Remainder Method

Uses the modulus operator to divide the input by the table size and take the remainder as the hash.

Multiplication Method

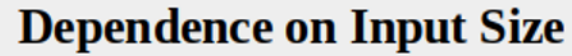
Multiplies the input by a constant, takes the fractional part of the result, multiplies this by the table size, and uses the integer part as the hash.

Folding Method

Splits the input into several parts, sums them, and then uses the modulus of this sum as the hash.

Mid-square Method

Squares the input, extracts a portion of the resulting number (often the middle part) to use as the hash.



Compiled by Dilip Maripuri



Data Structures

Hash Functions - Summary



Uniform Distribution	
Method	Description
Division-Remainder Method	Can be poor if the table size is not chosen well (e.g., not a prime number).
Multiplication Method	Typically provides good distribution, especially if the constant is chosen to promote incoherence between the data and the table size.
Folding Method	Reasonably good if the input is random and segments are appropriately combined.
Mid-square Method	Distribution can be uneven, depending on how digits are extracted from the squared result.



Data Structures

Hash Functions - Summary



Computational Efficiency	
Method	Description
Division-Remainder Method	Very efficient; simple modulus operation is all that is required.
Multiplication Method	Moderately efficient; involves multiplication and floating-point operations, which are costlier than simple modulus.
Folding Method	Can be less efficient due to multiple additions and possibly more complex data handling.
Mid-square Method	Requires more computation (squaring and digit extraction), making it potentially the least efficient of the four.



Data Structures

Hash Functions - Summary



Collision Handling

Method	Description
Division-Remainder Method	Collisions likely if the modulus is not well chosen or data is not uniformly distributed.
Multiplication Method	Collisions are generally fewer due to better spread, especially with a well-chosen multiplier.
Folding Method	Collisions depend on how the data segments overlap and are combined.
Mid-square Method	Potential for more unique collisions due to reliance on certain parts of the squared number.



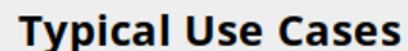
Data Structures

Hash Functions - Summary



Ease of Implementation

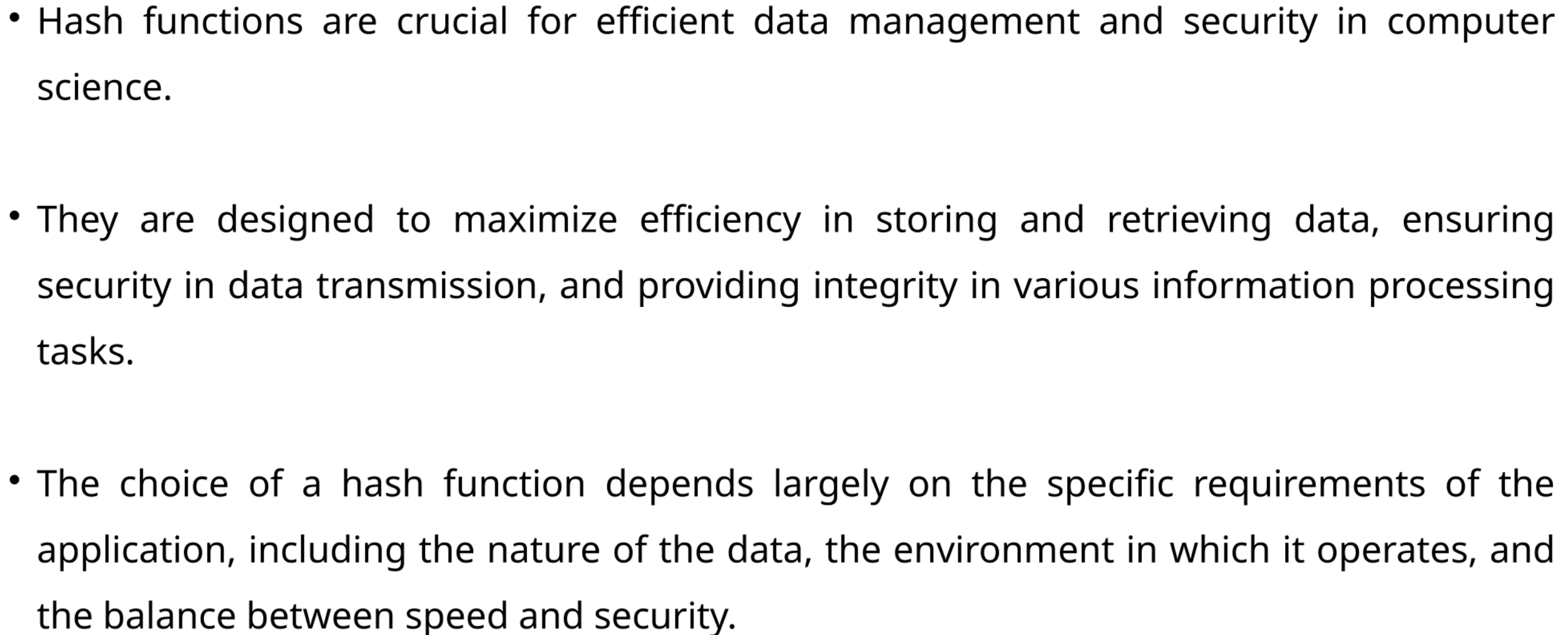
Method	Description
Division-Remainder Method	Very easy to implement.
Multiplication Method	Moderate difficulty due to the need to manage fractional parts and constants.
Folding Method	Moderately complex, requires careful handling of data segments.
Mid-square Method	More complex due to the need for squaring and digit extraction procedures.



Compiled by Dilip Maripuri



2014 2013 13 12 11 10 9 8 7 6 5 4 3 2 1 0





THANK YOU

Dilip Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu